In this project, I develop a Python program `idoc_scanner.py` which transforms a distorted document into a flat “top-down” view.

Its usage:

```
python idoc_scanner.py distorted.jpg restored.jpg
```

In the following, I would introduce my method and analyze whether preferrable results are produced.

I divide the whole program into 4 different stages. They are image pre-processing, finding vertices of the document, image wrapping and image post-processing.

In the .py file, numpy, cv2 and sys are imported.

1. Image Pre-processing

```python
def main():
    if len(sys.argv) != 3:
        print("Usage: python idoc_scanner.py <input image> <output image>")
        sys.exit(0)

    black = resize_for_display(np.zeros((imageHeight, imageWidth, 3), np.uint8))

    # IMAGE IMPORT
    image = cv2.imread(sys.argv[1])
    if image is None:
        print(f"Error: Could not find image {sys.argv[1]}")
        sys.exit(1)

    # IMAGE PREPROCESSING
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred = cv2.medianBlur(gray, 55) # VERY BLUR TO SHOW ONLY DOCUMENT EDGE
    edges = cv2.Canny(blurred, 150, 255)
    kernel = np.ones((5,5))
    dila =cv2.dilate(edges, kernel, iterations=2)
    edges =cv2.erode(dila, kernel, iterations=1)
```

Firstly, we check whether the command contains 3 arguments, and then we import the image. The `black` variable is for debugging at the end of program. Then we preprocess the image for easier contour finding. First we make the image gray and blurred so that the Canny function can be applied. In `medianBlur` function, I apply a large block size of 55 to the image such that the `Canny` function can ignore all the small neighborhood changes of pixels and focus on document edge. I further apply dilation and erosion to denoise the image.

2. Finding Contour and Vertices of the document

```python
# DRAW CONTOUR IN IMAGE
image_all_contour = image.copy()
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(image_all_contour, contours, -1, (0, 255, 0), 10)

# FIND LARGEST CONTOUR AND VERTICES
max_area = 0
biggest = None
for i in contours:
    area = cv2.contourArea(i)
    if area > 5000:
        temp = cv2.approxPolyDP(i, 0.02 * cv2.arcLength(i, True), True)
        if area > max_area and len(temp) == 4:
            biggest = temp
            max_area = area
if biggest is None:
    print("No document found! Using entire image as fallback.")
    h, w = image.shape[:2]
    # Create corners for entire image
    biggest = np.array([[[0, 0]], [[w, 0]], [[w, h]], [[0, h]]], dtype=np.int32)
    warped = cv2.resize(gray, (imageWidth, imageHeight))  # Just resize the grayscale image
```

Now we use cv2's function `findContours` to find the contour of the processed image. The variable `image_all_contour` is for debugging use. Since there may be contours inside the scanned document, we try to find the largest contour such that the variable `biggest` contains 4 vertices of the scanned document only. I find the area of the bounded contour to check whether maximum area is achieved after going through all contours detected. After that, if no vertices found, which means we could not find any 4 vertices such that it attains the maximum area. We don't want to warp it as a weird polygon with vertices not equal to 4, so assume no document is found, and we will skip the warping part and go post-processing.

3. Image Wrapping

```
# REORDER VERTICES FOR WRAP
biggest = biggest.reshape((4,2))
temp = np.zeros((4,1,2), dtype=np.int32)

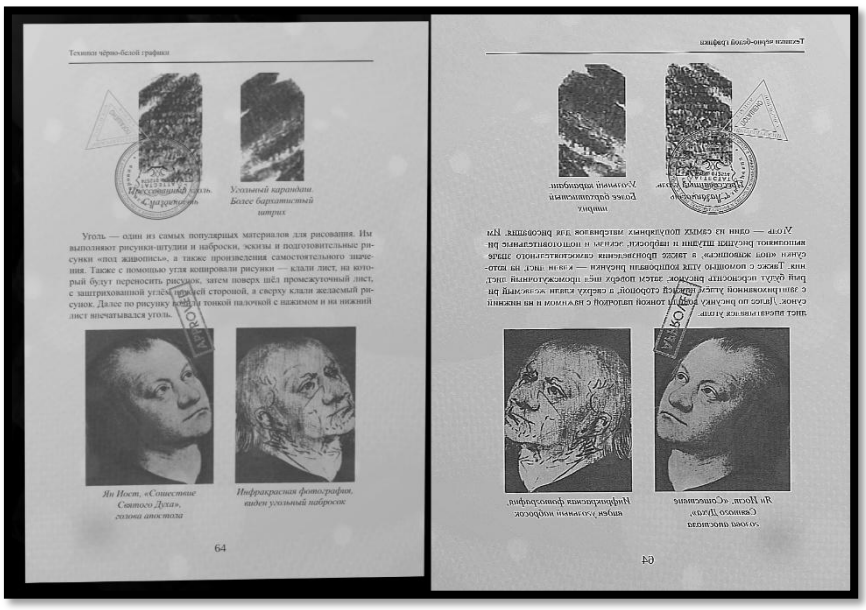
add = biggest.sum(1)
temp[0] = biggest[np.argmin(add)]
temp[3] = biggest[np.argmax(add)]

diff = np.diff(biggest, axis=1)
temp[1] = biggest[np.argmin(diff)]
temp[2] = biggest[np.argmax(diff)]

biggest = temp

# WARPING
if biggest.size != 0:
    points1 = np.float32(biggest)
    points2 = np.float32([[0,0],[imageWidth,0],[0,imageHeight],[imageWidth,imageHeight]])
    matrix = cv2.getPerspectiveTransform(points1, points2)
    warped = cv2.warpPerspective(gray, matrix,(imageWidth, imageHeight))
# HERE WARPED USES GRAY FOR POSTPROCESSING, NORMALLY DIRECT USE IMAGE
```

We first reorder vertices for warping since it must be in the order “left upper”, “right upper”, “left lower”, “right lower”. Else, the output image may be laterally inverted, or upside down, like the following.



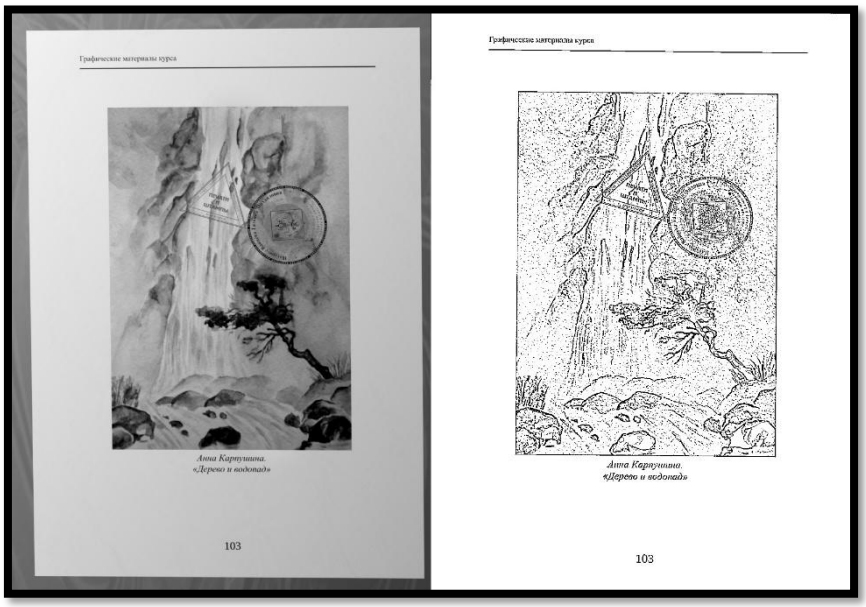
Then we use warp function in cv2 to warp original biggest variable to the new points.

4. Image post-processing

```
# IMAGE POSTPROCESSING
result = warped.copy()
"""
result = cv2.adaptiveThreshold(result, 255, 1, 1, 3, 2)
result = cv2.bitwise_not(result)
"""
I_sharpen = np.array([[-1,-1,-1],
                      [-1, 9,-1],
                      [-1,-1,-1]])
result = cv2.filter2D(result, -1, I_sharpen)
```

Here, we will sharpen the resulting image by the matrix learnt from the course, and uses cv2's filter function to apply the matrix to the output image.

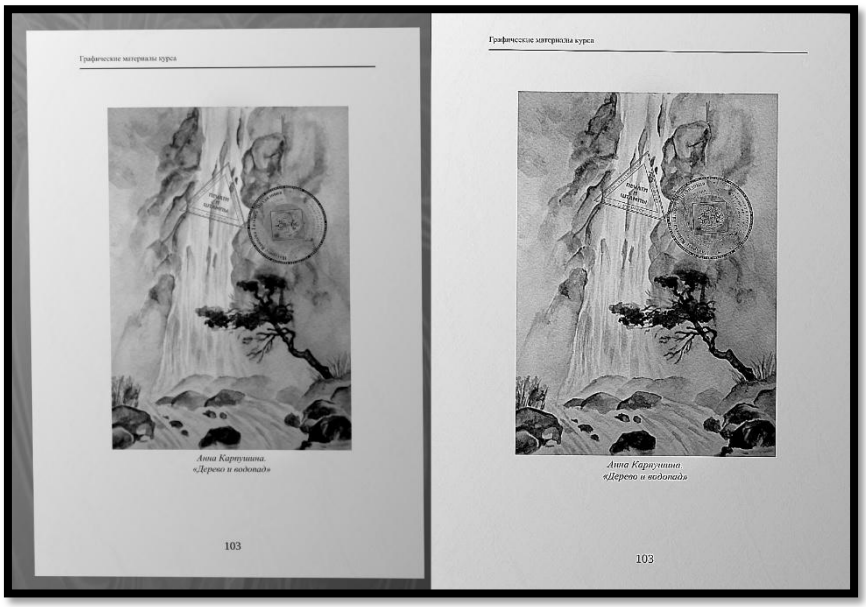
Since in the expected output in project description page, the output image background is of white color, I tried to use `adaptiveThreshold` function such that if the pixels' greyscale number is high, then we turn it into 255, which is white color. Nonetheless, it will output such image as follows.

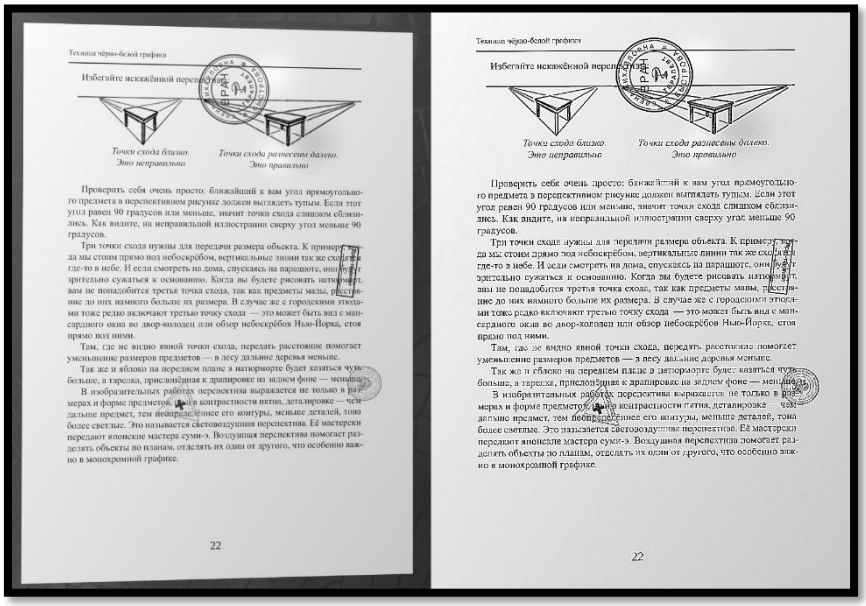


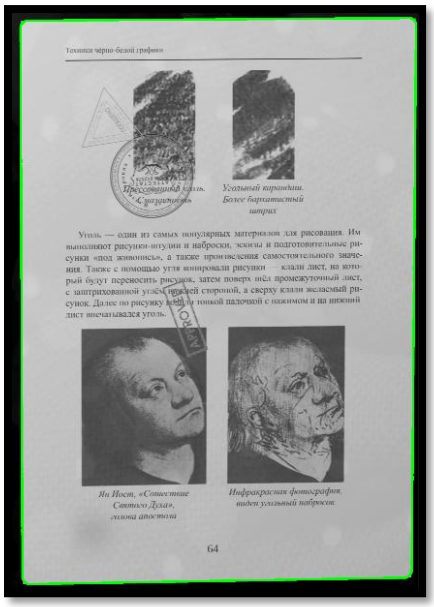
Above, the image inside the document is altered, which is not expected. Our goal is to preserve the photo inside the document unchanged. Therefore,

sharpening function is the only function that is applied on the image, even though white background is not output as expected.

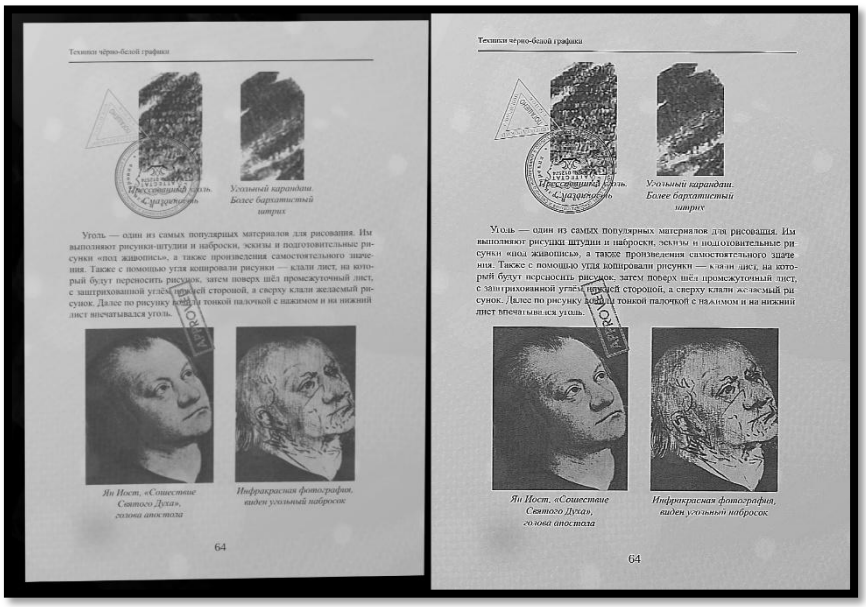
Here are some examples of successful warping. The left one is the distorted image, while the right one is the restored image.



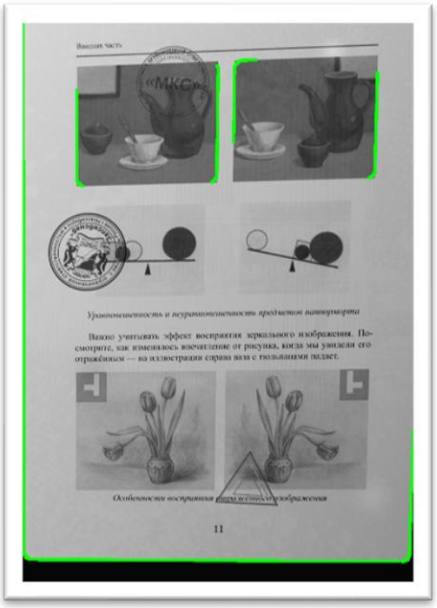




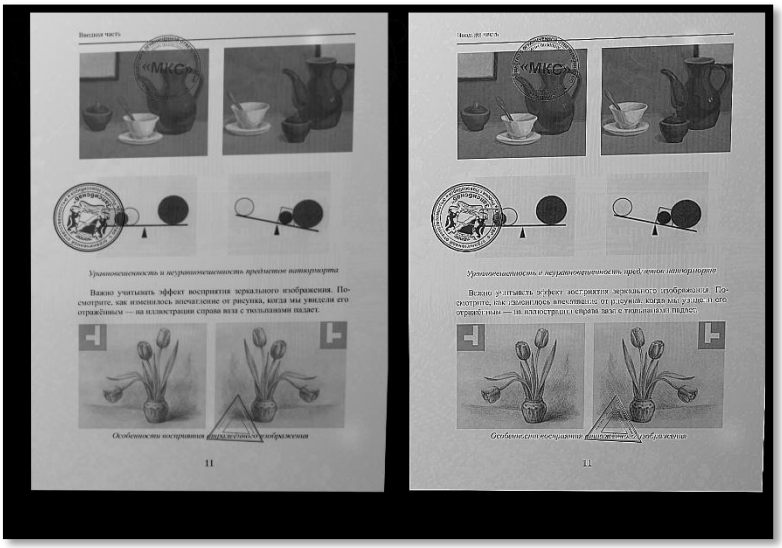
Above, the green lines are drawn. We can see that `drawContours` function locates the correct largest contour in the input image. Since 4 vertices are found, we can easily warp it into a scanned document.



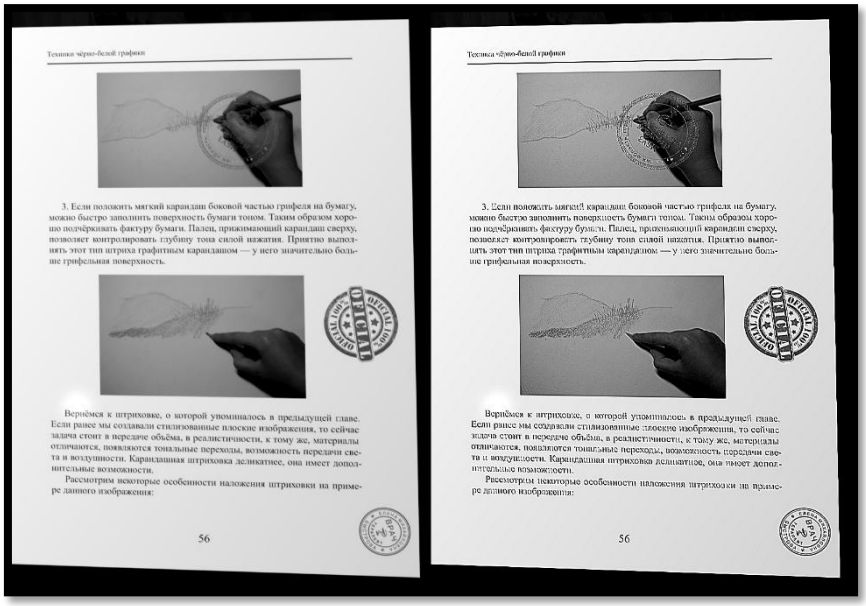
There are also some images which are not successfully warped. See below.

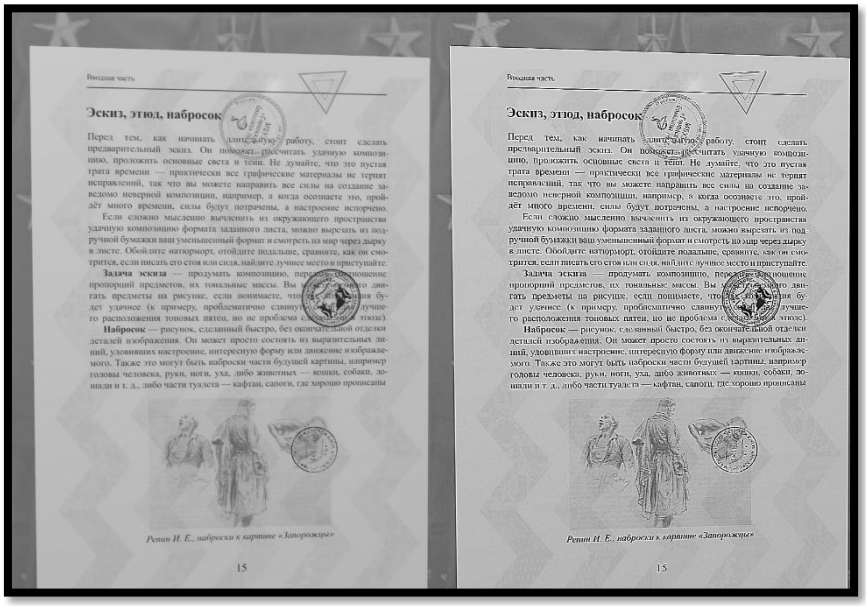


From above, we see that the found contours are not connected. Also, the upper and right edge of the document is cropped by the image, then the `cv2.findContours` function could not locate the contour accurately. Since my program directly skip the warp part of the image, we could only get the sharpened version of the original image.



Other failed examples are shown below.



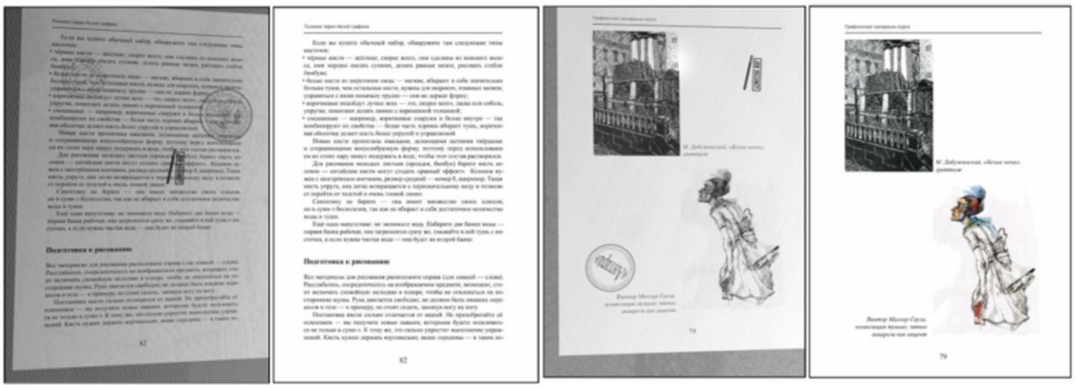


5. Limitations

Moreover, there are limitations in the implementation of the program.

Firstly, we cannot handle cases where corners or edges of scanned document is being cut, i.e. a document of 5 vertices, or a document of shape of trapezoid.

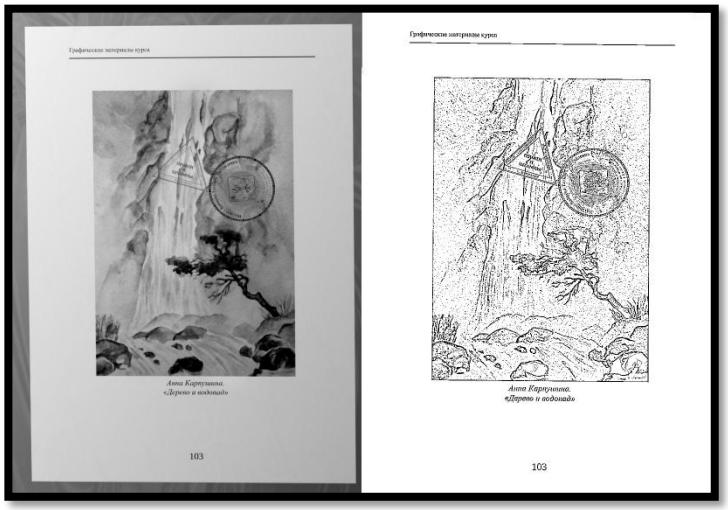
Secondly, we are unable to largely modify input image.



From project description, we can see that the program should be able to convert images from grayscale to colored version, as well as remove the symbols and irrelevant markings on the document, which our program is unable to accomplish.

In addition, our background of scanned documents may not be fully white if we don’t want to alter the images inside the document. If there is no image inside, the `cv2.adaptiveThreshold` function does a very good job of turning the

image into a document-like image. Below are some comparisons between original image and images that uses `cv2.adaptiveThreshold` function.



.

